

From Scratch:

The Design and Implementation of a SAT Solver From Scratch

Oliver Lie

July 26, 2026

1 Abstract

This paper discusses the design and implementation methods of a SAT solver. It explores... INSERT METHODS HERE

2 Introduction

SAT solvers are amazing programs with one simple purpose: solve boolean algebra equations quickly.

2.1 CNF

CNF stands for "Conjunctive Normal Form," sometimes called "Clausal Normal Form." An equation in this form is a "conjunction of disjunctions," ie, it is an equation of "ands" ($\wedge$) in between clauses of "ors" ($\vee$). Each clause of "ors" contains one or more literals, or variables. Each literal has the option to be negated ($\neg$), however, no negation can appear in front of a clause itself. Some examples of CNF equations includes:

- $(A \vee B)$
- $(A \vee B) \wedge (\neg C)$
- $(A \vee B) \wedge (\neg C \vee D)$

Some examples that are not CNF:

- $\neg(A \vee B)$, where there is a NOT in front of a clause
- $\neg(A \vee B) \wedge (C)$, where there is a NOT in front of a clause
- $(A \vee B \wedge C)$, where there is an AND within a clause
- $(A \wedge B) \vee (C)$, where it is in DNF form (read ahead)

2.2 DNF

DNF stands for "Disjunctive Normal Form." Just like in CNF form, a DNF equation has only three operators: conjunction ($\wedge$), disjunction ($\vee$), and negation ($\neg$). A DNF equation is in the form of a "disjunction of conjunctions," ie, it is an equation of "ors" ($\vee$) in between clauses of "ands" ($\wedge$). Each clause of "ands" contains one or more literals, each with the option of being negated ($\neg$), with no negation being in front of the clauses themselves. Some examples of DNF equations include:

- $(A \wedge B)$
- $(A \wedge B) \vee (C)$
- $(A \wedge B \wedge C) \vee (D \wedge E)$

Some examples that are not DNF:

- $(A \vee B)$, this is CNF form
- $\neg(A \wedge B) \vee (C)$, where there is a NOT in front of a clause
- $(A \wedge B \vee C) \vee (D \wedge E)$, where there is an OR within a clause

2.3 Defining our Input

For this SAT solver, we will use the DIMACS CNF format of input. DIMACS CNF format consists of seven rules:¹

1. The file may begin with comment lines. The first character of each comment line must be a lower case letter "c". Comment lines typically occur in one section at the beginning of the file, but are allowed to appear throughout the file.
2. The comment lines are followed by the "problem" line. This begins with a lower case "p" followed by a space, followed by the problem type, which for CNF files is "cnf", followed by the number of variables followed by the number of clauses.
3. The remainder of the file contains lines defining the clauses, one by one.
4. A clause is defined by listing the index of each positive literal, and the negative index of each negative literal. Indices are 1-based, and for obvious reasons the index 0 is not allowed.
5. The definition of a clause may extend beyond a single line of text.
6. The definition of a clause is terminated by a final value of "0".
7. The file terminates after the last clause is defined.

There are some things about this syntax to be wary of:

1. The definition of the next clause normally begins on a new line, but may follow, on the same line, the "0" that marks the end of the previous clause.

¹<https://people.sc.fsu.edu/~jburkardt/data/cnf/cnf.html>

2. In some examples of CNF files, the definition of the last clause is not terminated by a final '0'
3. In some examples of CNF files, the rule that the variables are numbered from 1 to N is not followed. The file might declare that there are 10 variables, for instance, but allow them to be numbered 2 through 11.
4. Comments can appear anywhere in the file, but the comment must be on its own line and it must start with 'c' ²

For a quick example, here is a simple way to format a DIMACS CNF equation:

```
p cnf 4 2
1 2 - 3 0
-1 4 0
```

And one with comments:

```
c this is a comment
p cnf 4 2
c this is another comment 1 2 - 3 0
-1 4 0
```

2.4 Input Processing

In order to process our input, we will provide a few APIs. We will firstly, disallow the header line. Ie, if no header is passed in, rather than throwing an error, we will simply infer the number of variables and clauses. We will also be very loose in terms of what input is considered valid. Our rules will be as follows:

1. Comments can appear anywhere in the file, but they **MUST** be on their own line and start with 'c'.
2. A header is not necessary, and if one is not provided, then the number of variables and clauses shall be inferred. However, if one is provided, then the number of variables and clauses **MUST** match the header.
3. All clauses, with the exception of the last clause, must end with '0', and can extend multiple lines.
4. All variables are numbered 1 - N, where N is the number of variables. Ie, if you say 10 variables, they must be labeled 1, 2, ..., N. And not 2, 3, ..., 11 or any other convention.

Thus, valid input can take on many forms:

```
p cnf 4 2
1 2 - 3 0
-1 4 0
```

²<https://jix.github.io/varisat/manual/0.2.0/formats/dimacs.html>

```

4 2
1 2 - 3 0
-1 4 0

```

```

p cnf 4 2
c this comment doesn't have to be at the top
1 2 - 3 0
-1 4 0

```

```

p cnf 4 2
c this comment doesn't have to be at the top
1 2 - 3 0
-1 4

```

```

c the first two numbers are still the number
c of variables and the number of clauses
p cnf 4 2 1 2 - 3 0
-1 4 0

```

Now that we have fully defined our input rules, we can now define our API.

```

class SATSolver: public interface
    SATSolver(string input);
    int getNumVars();
    int getNumClauses();
    vector<vector<int>> getClauses();
    optional<vector<bool>> getSolution();
    void printSolution();
    bool solveCNF();
class SATSolver: private interface
    void parse(string equation);
    int numVars;
    int numClauses;
    vector<vector<int>> clauses;
    optional<vector<bool>> lits;

```

Our API takes in a string, either representing a file or a CNF equation and parses it into memory for our usage.

Pseudocode:

```

SATSolver(string input):
    lits ← no-value
    if (input is file) then
        string content ← readFile(input)
        parse(content)
    else
        parse(input)
void parse(equation):
    if (contains(equation, "p cnf")) then
        int[0...n] A
        for each line in input:

```

```

        if (startsWith(line, 'c')) then
            skip to next line
        else
            for each token in line:
                A ← A::token
numVars ← A[0]
numClauses ← A[1]
idx ← 2
for i ← 0 to numClauses:
    int[0..n] clause
    while A[idx] ≠ 0
        clause ← A[idx]
        idx ← idx + 1
    clauses ← clause
else
    numVars ← 0
    numClauses ← 0
    int[0..n] A
    for each line in input:
        if (startsWith(line, 'c')) then
            skip to next line
        else
            for each token in line:
                numVars ← max(token, numVars)
                if token = 0 then
                    numClauses ← numClauses + 1
                A ← A::token
    idx ← 0
    for i ← 0 to numClauses:
        int[0..n] clause
        while A[idx] ≠ 0
            clause ← A[idx]
            idx ← idx + 1
        clauses ← clause

```

While this algorithm isn't the most efficient, it is however readable enough to the point where anyone can understand what it's doing. `SATSolver(...)` checks if the input is a file, and if so, it reads in the file contents, then parses it as a CNF equation. Otherwise, it treats it as a CNF equation and then parses it as one. Then `parse(...)` extracts every numeric token that is not on a comment line, then it uses that numeric data to initialize its variables and data structures. It should be noted that this pseudocode does not handle bad inputs like a negative number of inputs and clauses, as it assumes perfect input. The actual implementation is up to you and me. To see how it was implemented in actual code, you are more than welcome to read the (commented since C++ isn't the most readable at times) source code.

3 Brute Force

The absolute simplest way of solving a SAT problem. In this section, we create our base SAT solver from scratch. We will not add any fancy features such as incremental solving, backtracking, or even basic contradiction detection... at first. In our simple brute force solver, we are given an input as defined above, and we simply solve by checking every possible combination of variable assignment. The pseudocode for this is actually just as simple as it sounds:

```
bool solveCNF():
    for each set of literal assignments:
        bool solved ← true
        for each clause in clauses:
            bool clause_true ← false
            for each literal_assignment in clause:
                clause_true ← clause_true or literal_assignment
            solved ← solved and clause_true
        if solved = true then
            lits ← current_set_of_literals
            return solved
    return false
```

Why this code works should be obvious. We generate up to 2^n sets of literals assignments (where $n = \text{numVars}$) then we check each one against our CNF equation. A clause is true if any one of the literals in the clause evaluates to true, and is false otherwise. And the whole CNF equation is true if every clause has at least one literal whose assignment within the clause is true. In that case, we store that solution, and return true, and in the case where none of the 2^n sets of literals assignments leads to a solvable outcome, we return false. A runtime analysis of this version of the algorithm shows that in the worst case (the equation is unsatisfiable and every combination must be checked) the algorithm runs in $O(2^n ms)$, where 2^n represents the number of combinations we need to check, m is the amount of time it takes to evaluate the whole equation, and s represents the amount of time it takes to generate one of the combinations that we check. This means in the worst case, the algorithm is abysmally slow. But what about the best case? Well, in the best case, our first literal assignment is the solution, so in that case it will run in $\Omega(ms)$. However, the likelihood of the best case ever happening is abysmally low, and the algorithm should be considered to always run in its worst case time.

Once again, you are welcome to read the source code for the brute force solver (with comments).

3.1 Brute Force Results

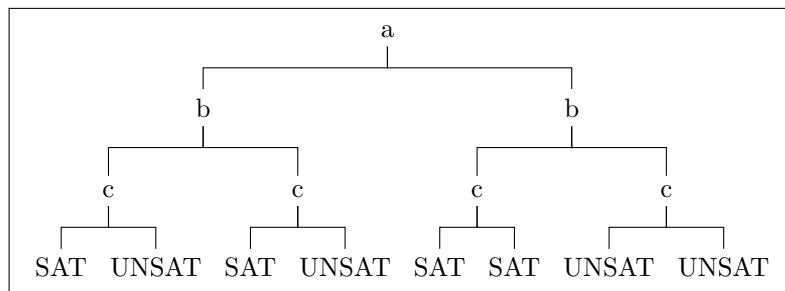
To see how well our current algorithm does, running some tests would be a good idea. Why run tests when we have already proven that our algorithm works "perfectly?" To see how quickly it runs of course. To test its speed (and as we add more features, to ensure correctness is kept), we use a database of problems from here: <https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html>. We gave our current implementation 1000 SAT instances (uf20-91), each with

20 variables and 91 clauses each, all satisfiable. It took this baseline brute force algorithm 2,119,301 milliseconds, which is roughly 35.32 minutes. Normally in science you have to run an experiment at least three times for data accuracy. But you must understand that I don't want to give up an hour and a half of my life to an algorithm that I'm going to be improving upon anyways.

3.2 Recursive Backtracking Search

4 DPLL

The DPLL algorithm, or the Davis-Putnam-Logemann-Loveland algorithm, is a complete, backtracking-based search algorithm for deciding whether or not a CNF problem is satisfiable or not. How does it work? Imagine a binary tree of variables, each with two branches representing their assignment, one for *true* and one for *false*. Suppose we don't know anything about our CNF equation. Ie, from looking at it, any literal could have any value, and we don't know how each assignment helps us in finding out whether or not the equation is satisfiable or not. How can we figure out if we can satisfy the equation or not without just brute forcing it? The answer is ~~making an ass out of you and me~~ to make assumptions.



The above picture of a DPLL decision tree (the outcomes may not map to a CNF problem, sue me).

Going back to our hypothetical, we don't know anything about anything. We could have a giant decision tree to explore with an exponential number of leaves to evaluate. How does making assumptions help us? Do we just run a random number generator and if it returns 1, we go "okay it's satisfiable" and false otherwise? No. We make assumptions about our literal assignments. Unlike brute forcing where we try every possible combination of variable assignments at once, in DPLL, we start with a literal, we'll call it *A*. There is nothing in the equation that indicates whether or not *A* should be true or false, either could be valid. So to tackle our problem, initially, we will assume *A* is true and see where that leads us. We will propagate that value throughout our clauses, and suppose there are two clauses where one forces *A* = *false* and the other forces *A* = *true*. That's clearly a contradiction, so we've figured out that at the very least *A* = *true* leads to a contradiction. So now, we've eliminated half of the search space immediately and we only need to consider the cases where *A* = *false*. This may not make total sense immediately, so instead of giving one explanation of the algorithm and building it, we will build up the algorithm and explain how each part works.

4.1 Recursive Backtracking

The first piece of this algorithm is a method that will recursively assign each literal's value as it traverses down the decision tree. This means we will need to change our private API to suit a new recursive method.

```
class SATSolver: public interface
    SATSolver(string input);
    int getNumVars();
    int getNumClauses();
    vector<vector<int>> getClauses();
    optional<vector<bool>> getSolution();
    void printSolution();
    bool solveCNF();
class SATSolver: private interface
    void parse(string equation);
    int numVars;
    int numClauses;
    vector<vector<int>> clauses;
    optional<vector<bool>> lits;
```

The old interface.

```
class SATSolver: public interface
    SATSolver(string input);
    int getNumVars();
    int getNumClauses();
    vector<vector<int>> getClauses();
    optional<vector<bool>> getSolution();
    void printSolution();
    bool solveCNF();
class SATSolver: private interface
    void parse(string equation);
    bool solve(vector<bool> assignment);
    int numVars;
    int numClauses;
    vector<vector<int>> clauses;
    optional<vector<bool>> lits;
```

The new interface, where *solve()* is a recursive method.

The pseudocode is as follows:

```
bool solveCNF():
    solve(lits)
bool solve(vector<bool> assignment[0...n]):
    if assignment.size == numVars then
        bool solved ← true
        for each clause in clauses:
            bool clause_true ← false
            for each literal_assignment in clause:
                clause_true ← clause_true or literal_assignment
```



```

        solved ← solved and clause_true
    return solved
else
    assignment ← assignment::true
    if solve(assignment) then
        return true
    else
        assignment ← assignment[0...n-1]
        assignment ← assignment::false
        if solve(assignment) then
            return true
        else
            assignment ← assignment[0...n-1]
            return false

```

4.2 Recursive Backtracing - Results

Running this code on the same 1000 instances (uf20-91), it took 1846644ms, which is roughly 30.78 minutes. This is actually a pretty big improvement from our 35.32 minutes via brute forcing. Although this seems like a big gain in speed, it should be noted that at this point the algorithm is still just a fancier brute forcing algorithm.

4.3 Unit Clause Propagation

After recursive traversal and backtracing, unit clause propagation is the next big piece of our "assume and explore" algorithm. Here's why it's useful: suppose you have a CNF equation like this: $(A) \wedge (A \vee B)$, or $(A) \wedge (\neg A \vee B)$. Notice how in order for the equation to be satisfied, A MUST be true by virtue of simply being by itself. Because of this, in the first equation, we see that B can either be true or false because just A being true is enough to satisfy everything. But in the second equation, $A = \text{true}$ FORCES $B = \text{true}$ as well, because $\neg A$ can't be true. On these small examples, this revelation doesn't seem important, but it's very important on larger examples, it can make a massive difference in speed.

But what if we don't have any unit clauses? Suppose we have the equation: $(A \vee B) \wedge (\neg A \vee C)$. There are no unit clauses, so we might think that we can't propagate anything. However, what if we assumed something was true? Visually speaking, since A appears twice in the equation across both clauses, A seems like a good pick to try this on. So let's assume $A = \text{true}$. What does that tell us? Well it immediately tells us that B can take on any value, but C must be true as well because $\neg A = \text{false}$. So now, in this slightly larger example, instead of brute forcing 8 cases (because 2^3 instances), we made 1 assumption and from there deduced one satisfiable assignment to the equation ($A = \text{true}, B = \text{false}, C = \text{true}$).

Now that we've seen the power of assuming (don't do this in real life), let's modify our API and define pseudocode. We will add a new method for learning and assuming. To do this, we need to add in a new field as well.

```

class SATSolver: public interface
    SATSolver(string input);
    int getNumVars();
    int getNumClauses();
    vector<vector<int>> getClauses();
    optional<vector<bool>> getSolution();
    void printSolution();
    bool solveCNF();
class SATSolver: private interface
    void parse(string equation);
    bool solve(vector<bool> assignment);
    int numVars;
    int numClauses;
    vector<vector<int>> clauses;
    optional<vector<bool>> lits;

```

The old interface.

```

class SATSolver: public interface
    SATSolver(string input);
    int getNumVars();
    int getNumClauses();
    vector<vector<int>> getClauses();
    optional<vector<bool>> getSolution();
    void printSolution();
    bool solveCNF();
class SATSolver: private interface
    void parse(string equation);
    bool solve(vector<bool> assignment);
    vector<int> learn();
    int numVars;
    int numClauses;
    set<int> learned;
    vector<vector<int>> clauses;
    optional<vector<bool>> lits;

```

The new interface.

The idea of *learn()* is that we have a set of learned literal values and from that set of learned values, we want to deduce the values of other literals based off of what we know or assume. The easiest way to learn the value of a literal is to encounter a unit clause. A unit clause is a type of clause that contains only one literal, and because in CNF each must be satisfied, in order to satisfy the unit clause, the literal must have the value in the clause. So if we encounter 1, then literal 1 must be true. Likewise, if we encounter -1 , -1 must be true, which means 1 must be false. After figuring out the value of 1, suppose we have the clause -12 , and let's suppose $1 = \text{true}$. Because we learnt that $1 = \text{true}$, that means $-1 = \text{false}$, therefore, $2 = \text{true}$ must be true in order to satisfy that clause. Now suppose we the clause 12 and $1 = \text{true}$. What can we learn about 2? The answer is nothing! Because the clause is already satisfied by 1, nothing that we know forces 2 to be true or false, so we can't deduce anything.

So you can see how learning and propagation works. Before we modify the code of *solveCNF()* and *solve()*, we will see what the pseudocode for *learn()* is.

The pseudocode for *learn()* is as follows:

```

vector<int> learn():
    set<int> learnt
    vector<bool> indices
    curSize ← learned.size()
    newSize ← curSize + 1
    while curSize ≠ newSize:
        curSize ← learned.size()
        for i ← 0 to clauses.size():
            if indices[i] ≠ true:
                at_least_one_true
                if clauses[i].size() = 1 and learned.contains(clauses[i][0]) = false then:
                    learnt.insert(clauses[i][0])
                    learned.insert(clauses[i][0])
                    indices[i] = true
                else
                    num_not_learnt ← 0
                    else_false ← false
                    index ← 0
                    for j ← 0 to clauses[i].size():
                        was_learned ← learned.contains(clauses[i][j]) or learned.contains(-clauses[i][j])
                        if was_learned = false and abs(clauses[i][j]) ∉ lits.size() then:
                            index = j
                            num_not_learnt ← num_not_learnt + 1
                            if num_not_learnt ∉ 1 then break
                    else:
                        forced_val
                        if was_learned and clauses[i][j] ∉ 0 then:
                            forced_val ← not learned.contains(clauses[i][j])
                        else if was_learned and clauses[i][j] ∉ 0 then:
                            forced_val ← learned.contains(clauses[i][j])
                        else forced_val ← lits[clauses[i][j]]
                        eval
                        if clauses[i][j] ∉ 0 then: eval ← not forced_val
                        else: eval ← forced_val
                        if eval = true then at_least_one_true = true
                    if num_not_learnt = 1 and not at_least_one_true then:
                        learnt.insert(clauses[i][j])
                        learned.insert(clauses[i][j])
                        indices[i] = true
            newSize ← learned.size()
    return vector from learnt

```

This pseudocode is a mouthful. But essentially, *indices* keeps track of which clauses *learn()* knows are satisfied. Notice that it resets upon every call, so every time we want to learn something new, we need to check if it's already been learnt. If we encounter a unit clause, we immediately learn the literal's value, mark the clause as satisfied and continue. But in the non-unit clause case,

it's a bit harder to ensure that we can properly learn something new. The rules for a non-unit clause are: there is only one possible candidate to be learned, everything else has already been learned, and everything else within the clause evaluates to false. I.e., this literal's assignment **MUST** be true within the clause. So, all those conditionals in the else branch ensure beyond the shadow of a doubt that those conditions are met (I wrote the actual code when I was tired, and I'm making the pseudocode follow the actual code). (There is most definitely a more efficient encoding for what literals must be satisfied, and the learning algorithm could 100% be more efficiently written, but I'd rather have it work for now).

Now we can modify the pseudocode for *solveCNF()* and *solve()*:

```

bool solveCNF():
    learn()
    solve(lits)
bool solve(vector<bool> assignment[0...n]):
    if assignment.size == numVars then
        bool solved ← true
        for each clause in clauses:
            bool clause_true ← false
            for each literal_assignment in clause:
                clause_true ← clause_true or literal_assignment
            solved ← solved and clause_true
        return solved
    else
        cur_lit = assignment.size() + 1
        if learned.contains(cur_lit) then:
            assignment ← assignment::true
            if solve(assignment) then:
                return true
            else:
                assignment ← assignment[0...n-1]
                return false
        else if learned.contains(-cur_lit) then:
            assignment ← assignment::false
            if solve(assignment) then:
                return true
            else:
                assignment ← assignment[0...n-1]
                return false
        else:
            learned.insert(cur_lit)
            assignment ← assignment::true
            learned_lits ← learn()
            if solve(assignment) then:
                return true
            else:
                assignment ← assignment[0...n-1]
                assignment ← assignment::false
                learned.remove(cur_lit)

```

```

for i ← 0 to learned_lits.size()
    learned.remove(learned_lits[i])
learned.insert(-cur_lit)
learned_lits ← learn()
if solve(assignment) then:
    return true
else:
    learned.remove(-cur_lit)
    assignment ← assignment[0...n-1]
    for i ← 0 to learned_lits.size()
        learned.remove(learned_lits[i])
    return false

```

For this section of the paper especially I would really recommend looking at the actual implementation for more comments. But essentially, we modified our `solveCNF()` method to first learn what it can. Then at the time of variable assignment, we check if we've already learned that literal's value, and if we have, we assign it its learnt value, then try to solve. If it fails, we remove that assignment (we will discuss this further) and return false. Otherwise, we assume that literal is true, try to learn from that, ie see what that literal being true would imply for the other literals, then try to solve. If `solve()` returns false after that, we undo everything we learned, and we now assume that literal is false instead and repeat the process. In the case where this literal being false also ends up returning false, we unlearn everything once again and we return false as well.

So why is it that when we have learnt a literal's assignment, when evaluating the equation, if the current literal's learnt value doesn't lead to a satisfiable assignment of variables, we undo its assignment? This is because there are two types of learned values: ones that are forced (explicit), and ones that are inferred (implicit). What are some kinds of explicit learned values? They are literals whose values are learned from unit clauses, or literals whose values are inferred directly from unit clauses like in our example. What are some kinds of implicit learned values? The ones from assumed/inferred literals. Those ones need to be undone. How do we know that if we learn something explicitly it won't be unlearned or undone? Because if something was explicitly learned, it's done in `solveCNF()`, and therefore can't be undone on a new call because `solveCNF()` doesn't pass on the literals it learned onto `solve()`.

4.4 Unit Clause Propagation and Assumption - Results

After implementing unit clause propagation and assumptions into our DPLL algorithm, on the same 1000 instances of uf20-91, it took only 2363ms. This is a huge improvement. What used to take 30.78 minutes to solve now takes less than 2.5 seconds. As stated before, but may be forgotten, all 1000 problems in uf20-91 are satisfiable. But what about unsatisfiable problems? Before, we didn't test our SAT solver on any because our algorithm could only realize that a problem was unsatisfiable by testing every assignment. But now with unit clause propagation and assumption, we can figure out which branches could lead to a satisfiable assignment and which ones can't. Our second testing set is called UUF50.218.1000. It is a problem set of 1000 problems, each with 50

variables, and 218 clauses. Each problem is unsatisfiable. To solve just one using just recursive backtracking took over 10 minutes. However, with our new algorithm, to solve all 1000 cases, it took 646228ms, which is roughly 10.77 minutes. This again is a massive speedup, which makes testing our solver for both satisfiable and unsatisfiable cases viable, unlike before, which could only handle satisfiable cases.

4.5 Pure Literal Elimination

Suppose in our equation the literal $\neg 1$ appears in multiple places. Perhaps it doesn't appear in a large amount of clauses, but nowhere in the CNF do we ever encounter the literal 1. This is a pure literal, and they have the special property of being able to be assigned the value that will make them satisfactory within their clauses immediately. While it may seem intuitive, or perhaps even "duh," how can we be sure of this? Suppose we have this equation:

$$(1 \vee 2 \vee \neg 3) \wedge (1 \vee \neg 2) \wedge (1 \vee 3)$$

We see that 1 is a pure literal, more specifically, it's a pure positive literal. Why can we immediately assign $1 = \text{true}$? The answer is really simple: it doesn't hurt. Saying $1 = \text{true}$ obviously only helps us, we can visually see that, but since $\neg 1$ doesn't show up anywhere, $1 = \text{true}$ doesn't make any other clauses less satisfiable, it only makes certain clauses more satisfied, and that's why it's okay.

An interesting thing about pure literals is that it can lead to other pure literals. Suppose we had the equation:

$$(1 \vee 2) \wedge (\neg 2 \vee 3) \wedge (3 \vee 4)$$

1, 3, and 4 are all pure literals, but let's only focus on 1 right now. Assigning $1 = \text{true}$ satisfies the first clause, and so we don't need to consider it anymore in future calculations. Assigning $1 = \text{true}$ effectively turns the equation into:

$$(\neg 2 \vee 3) \wedge (3 \vee 4)$$

because the assignment of 2 is now irrelevant with respect to 1. Notice how $\neg 2$ is a pure literal now. So we can say that $2 = \text{false}$ (to satisfy $\neg 2 = \text{true}$). So by assigning one pure literal, we were able to propagate that too and figure out the rest of the equation without branching.

Now, one must remember that we should never assign something blindly. For instance, suppose we have the equation

$$(2) \wedge (1 \vee \neg 2) \wedge (3 \vee \neg 2)$$

because 2 is a pure literal, $2 = \text{true}$ MUST be true. And per our own rules from before, technically speaking $\neg 2$ is now a pure literal. Ignoring the fact that 1 and 3 are pure literals as well, it may be tempting to just assign $2 = \text{false}$ since it's less work than assigning to both 1 and 3, but that would be a contradiction because 2 has already been assigned. That's just something to be aware of, that the literal must not be already assigned in order to be assigned through pure literal elimination.

Without further ado, let us discuss pseudocode. We will add two new methods: one for marking clauses as satisfied, and one for doing the actual pure literal elimination.

```
class SATSolver: public interface
    SATSolver(string input);
    int getNumVars();
    int getNumClauses();
    vector<vector<int>> getClauses();
    optional<vector<bool>> getSolution();
    void printSolution();
    bool solveCNF();
class SATSolver: private interface
    void parse(string equation);
    bool solve(vector<bool> assignment);
    vector<int> learn();
    int numVars;
    int numClauses;
    set<int> learned;
    vector<vector<int>> clauses;
    optional<vector<bool>> lits;
```

The old interface.

```
class SATSolver: public interface
    SATSolver(string input);
    int getNumVars();
    int getNumClauses();
    vector<vector<int>> getClauses();
    optional<vector<bool>> getSolution();
    void printSolution();
    bool solveCNF();
class SATSolver: private interface
    void parse(string equation);
    bool solve(vector<bool> assignment);
    vector<int> learn();
    vector<int> mark_satisfied();
    vector<int> pure_lim_eliminate();
    int numVars;
    int numClauses;
    vector<bool> satisfied;
    set<int> learned;
    vector<vector<int>> clauses;
    optional<vector<bool>> lits;
```

The new interface.

The pseudocode for `mark_satisfied` is simple:

```
vector<int> mark_satisfied():
    vector<int> indices;
    for i ← 0 to numClauses:
```

```

    if satisfied[i] = false then
        for j ← 0 to clauses[i].size():
            if learned.contains(clauses[i][j]) then
                indices.push_back(i)
                satisfied[i] ← true
                break
return indices

```

Then for pure_lim_eliminate():

```

vector<int> pure_lim_eliminate():
    vector<int> pure;
    vector<bool> indices ← satisfied;
    map<int, unsigned int> counts;
    for i ← 0 to numClauses:
        if indices[i] = false then
            for j ← 0 to clauses[i].size():
                counts[clauses[i][j]] ← counts[clauses[i][j]] + 1
    while true:
        int num_pure ← 0
        for each [lit, count] in counts:
            if counts does not contain -lit and learned does not contain -lit then:
                pure.push_back(lit)
                learned.insert(lit)
                num_pure ← num_pure + 1
        if num_pure = 0 then:
            return pure
        for i ← 0 to numClauses:
            if indices[i] = false then
                bool is_satisfied ← false
                for j ← 0 to clauses[i].size():
                    if learned.contains(clauses[i][j]) then:
                        is_satisfied ← true
                        indices[i] ← true
                        break
                if is_satisfied = true then:
                    for j ← 0 to clauses[i].size():
                        counts[clauses[i][j]] ← counts[clauses[i][j]] - 1
        for each [lit, count] in counts:
            if count = 0 then:
                counts.remove(lit)
    return pure

```

What this code does is slightly less intuitive. First, we gather a count of each literal in the unsatisfied clauses. Then, for each unsatisfied clause, we check if it contains a literal that is both unlearned and whose negation is not contained in our counts, because if the negative of a literal does not appear counted, it is a pure literal. Then, if we encounter no pure literals, we exit the while loop, but if we do encounter at least one, we do the following: we reiterate over the

clauses, then find a clause that now has a pure literal within. And if it does have a pure literal, we reiterate over that specific clause and decrease the counts of all literals within that clause. Finally, we remove all elements from the counts map with a count of 0 because that way we have the chance to find more pure literals on the next iteration of the while loop.¹

4.6 Pure Literal Elimination - Results

After implementing pure literal elimination into our DPLL algorithm, on the same 1000 instances of uf20-91, it took 2165ms, which is only about a 200ms speedup from our last version. Honestly, this could be attributed to a lucky run, whether it be a good scheduler, warm cache, or the universe deciding that we deserve a 200ms speedup. More interestingly, on the unsatisfiable problems, it took 350027ms (5.83 minutes) to solve all 1000 instances. This is about a 50% speedup on the unsatisfiable problems, which took about 10.77 minutes without pure literal elimination. This is very nice, and while I can't speak for us, I am satisfied with the results as they are now. Luckily, there is more improvements to be done.

4.7 Literal Contradiction Detection

Literal contradiction detection is the process of detecting when a literal being assigned a certain value leads to a clause being unsatisfiable, or when a literal is forced to be both *true* and *false*. It's a very useful thing to check because it's almost like a means to "short-circuit" paths of the decision tree. The algorithm for literal contradiction detection is a part of unit propagation. So why didn't we add it sooner? Because I don't know, that's why. And, since I'm tired, and this isn't going to be the final paper anyways, I'm not writing down the pseudocode.

4.8 Literal Contradiction Detection - Results

After implementing literal contradiction detection into our DPLL algorithm, on the same 1000 instances of uf20-91, it took 1477ms, which roughly 700ms of speedup from our last version, and in case you need a reminder, the last version only gave us 200ms of speedup, which could have just been operating system scheduler luck. And, in a much more impactful speedup, on the unsatisfiable problems, it took 121870ms (2.03 minutes) to solve all 1000 instances. This is very nice, not only because the unsatisfiable problem set is much larger than the satisfiable set (50 literals and 218 clauses vs 20 literals and 91 clauses), unsatisfiable problems are much harder to show unsatisfiable, so having any significant speedup on them is very welcome. Although, because of the nature of the code being used for teaching purposes and conceptualization, there are some places that aren't running as efficiently as they could be, either by Big-O, memory, or branching. This next portion of the paper will cover some optimizations we can make to our DPLL algorithm, although I don't expect a huge improvement in

¹In my implementation, I spent about 15 - 30 minutes debugging an infinite loop where pure literals weren't getting eliminated. The issue? I didn't initialize a loop variable, so it skipped the for loops entirely. This just goes to show that you can prove something works and then totally sputz it up in the actual implementation.

speed, but it will be good because I'm rushing to get this done before I graduate and I need to clean up my code anyways.

5 Optimized DPLL

Optimizing code is always a good thing. No matter what anyone says, whether it be "readability is more important," or "the hardware will speed it up for you," optimizing your code is always a good thing. Since these optimizations are largely language specific, you can skip to the next chapter when I read if you're not writing your solver in C++.

5.1 Analyzing Our Current Approach

If you look at the source code for the DPLL version of the SAT solver, you'll notice a pattern amongst `learn()`, `pure_lit_eliminate()`, and `mark_satisfied()`: copy the current state of the solver, make modifications, return the additions, then in `solve()`, we inspect what was returned, add them into the current solver state, and then continue solving. We want to optimize this process. Firstly, on large CNF equations, copying the learned literals and the solved clauses has some overhead, albeit not too much. Next, adding/removing the additions takes some work as well. But the biggest thing to optimize is how we evaluate the equation over and over and over. We evaluate it in `mark_satisfied()`, we evaluate parts of it `learn()`, in `pure_lit_eliminate()`, and every time we finish assigning all literals. This is especially wasteful on large equations, and it begs the question: since all three of these methods evaluate and in some way knows the state of the equation (solved or unsolved), why do we need to constantly reevaluate it? This is going to be the main focus of this chapter.

In order for our three learning methods to evaluate it more efficiently, we need to introduce some goals:

- No memory copies - we want to use and modify the permanent data structures within the helper methods.
- Short circuiting - if we find that the equation is already solved before we assign every literal, we should stop evaluation altogether.
- Easy undoing - when we inevitably go down a bad branch, undoing should be just as easy as doing.

Now would be a good time to introduce some invariants into our solver methods:

- *learned* is the set containing all learned literals, whether they be assigned (that means assumed literals are in there too), inferred, pure, or by some other means.
- *lits* contains the current assignment of our literals and literals are assigned strictly in order (1, 2, ..., n) (this will change eventually).
- *satisfied* contains which clauses are currently satisfied and which ones aren't at any given point in the solving state.

- *learn()* and *pure_{lit}eliminate()* mutate the solver state directly, and return any changes they made to the state.
- It is impossible for *learn()* and *pure_{lit}eliminate()* to learn a literal that was already learnt or satisfy a clause that was already satisfied.

Now that we've defined our invariants, the hard part now is enforcing each one! Luckily, the second invariant is already enforced within *solve()*, which leaves us with the other three to deal with. Fortunately, they're sort of intertwined, so dealing with one helps us deal with the other two at the same time.

5.2 Messy Optimizations and Proofs

This is going to have some messy, messy optimizations, and proofs of why they work. We will first change our API.

```
class SATSolver: public interface
    SATSolver(string input);
    int getNumVars();
    int getNumClauses();
    vector<vector<int>> getClauses();
    optional<vector<bool>> getSolution();
    void printSolution();
    bool solveCNF();
class SATSolver: private interface
    void parse(string equation);
    bool solve(vector<bool> assignment);
    vector<int> learn();
    vector<int> mark_satisfied();
    vector<int> pure_lim_eliminate();
    int numVars;
    int numClauses;
    vector<bool> satisfied;
    set<int> learned;
    vector<vector<int>> clauses;
    optional<vector<bool>> lits;
```

The old interface.

```

class SATSolver: public interface
    SATSolver(string input);
    int getNumVars();
    int getNumClauses();
    vector<vector<int>> getClauses();
    optional<vector<bool>> getSolution();
    void printSolution();
    bool solveCNF();
class SATSolver: private interface
    typedef learn_pair = pair<vector<int>, vector<int>>    void
    parse(string equation);
    bool solve(vector<bool> assignment);
    optional<learn_pair> learn();
    optional<vector<int>> mark_satisfied();
    learn_pair pure_lim_eliminate();
    int numVars;
    int numClauses;
    vector<bool> satisfied;
    set<int> learned;
    vector<vector<int>> clauses;
    optional<vector<bool>> lits;

```

The new interface.

We define a *learn_pair* as a pair of two vectors of ints. The first vector is ALWAYS the vector of learned literals, just like what the methods returned before. The second vector is the vector of clauses that were satisfied during the execution of the method. This allows us to be able to reduce memory copies of *learned* and *satisfied* because each method is returning a record of the modifications it made to both data structures.

5.3 Optimized DPLL - Results

After removing extra memory copies, reducing branching, and adding in a quicker evaluation techniques, it went from over 121870ms (2.03 minutes) for the unsat problems to 85440ms (1.42 minutes) and for the sat problems, it went from 1477ms to 1103ms. That is a huge improvement from optimizing my code in a way "oh yeah we should probably do that" way instead of a "let's perform a million micro-optimizations to get 1ms faster performance" way.

6 Watched Literals

For the next big architectural upgrade, we will introduce the notion of watched literals. Watched literals provide a more efficient way of propagating results throughout the CNF equation. In our old DPLL, whenever we change the value of a literal, we scan through every single clause in the equation, check if it's satisfied, check if that forces any other literals' values, etc. That's highly inefficient because we do so much assigning/unassigning that the runtime is actually mostly dominated by constantly reevaluating the whole equation. Intuitively speaking, watched literals are a method of only updating the clauses that were affected by the literal's value changing.

One's first thought for watched literals is most likely "let's create a map from literals to clauses, and when we update/force a literal's value, we mark some clauses as satisfied, check if it made any clauses unsatisfied, and if so, backtrack, otherwise continue." And that's a very reasonable first thought. And it certainly could lead to some performance gains. After all, it's a reasonable assumption that not every literal appears in every clause, so unless that happens, we need to scan much less! However, in the unreasonable case, we don't gain any performance because we must scan everything! What if there was a way to scan less, even in the most unreasonable/difficult cases?

This is where our second thought comes in. What if, conceptually, instead of having a map from literals to clauses, we have a map from clauses to literals? Currently, and in our previous example, each clause was watched by every literal within. And that is the bulk of our problem. What if we only updated the clause when certain literals were updated instead of any literal? How would we do that?

Suppose we have an equation:

$$(1 \vee 2 \vee 3) \wedge (\neg 2 \vee 4 \vee 5)$$

Let us ignore pure literal elimination, especially because in preparation for future improvements, we will be getting rid of it. In our normal case, when 1 updates, we scan each clause and ask "does it contain a 1?" And if it does, we update it accordingly. Then we move onto 2, then 3, then 4, then 5. We scanned the entire equation 5 times, evaluated it 5 times, and that wasn't even to see if our chosen assignment is satisfactory for the equation.

But suppose instead of tracking every variable we tracked only specific variables? Firstly, how would this work? For me, the biggest piece of confusion is "how do we know when to propagate?" And this is honestly the hardest thing to understand. Because when we update a literal, and we scan the whole equation, we know everything about the equation. We know what clauses are satisfied, which ones aren't, which variables need to be forced, if there's a contradiction, etc. We see everything because we scan everything and watch everything. So how do we keep track of all that without scanning everything? That's a big concern, and it's a very good one to have. Think about this: when are we in danger of having an unsatisfiable assignment? The answer is obvious, we're in danger of having an unsatisfiable assignment when almost everything in a clause is false, ie when there's less and less in the clause that could potentially satisfy it.

So following up on that, how do we know when a clause can't be satisfied? When no literal in the clause evaluates to true. Duh! So suppose we weren't analyzing every literal, how would we know when a clause can't be satisfied? When the literal(s) we're looking at in the clause are false, and there are no replacements! And how do we know when there are no replacements? Counter-intuitively, by scanning the entire clause! That's confusing.

Instead of deriving the whole algorithm, perhaps it's best to explain it. Every clause has at least one watched literal, unless it's an empty clause. In that case, the equation is unsatisfiable, so do nothing, just return false. When does a clause have two watched literals vs one watched literal? A clause has one watched literal iff it only has one literal within it. If a clause has two or more literals, then it has exactly two. Every literal has three states: True, False, and

Unassigned. And a literal can only be watched when it is either Unassigned or True (as evaluated within the clause). What happens when a watched literal is evaluated to False within a clause? We try to swap it out with a different literal that either evaluates to True or is Unassigned. If we are able to do that, then the clause can still be satisfied, otherwise, we arrive at a contradiction and have to backtrack.

But what about unit propagation? How do we ensure that we're still able to force literals in other clauses, if possible? We use a second watched literal. What? Seriously? How does that help? Well, we assign literals one by one, no matter what order, we assign them one by one. And each watched literal either evaluates to True, or it is Unassigned. Because of this invariant, if both watched literals are Unassigned, the clause is either satisfied or unsatisfied. If one watched literal is True, the clause is satisfied, no matter what the value of the other one is. If both of them are False, then the whole clause must be unsatisfied BECAUSE we couldn't find another literal in the clause that is either True or Unassigned. What about when one is False, and the other is Unassigned? In that case, when one watched literal is False and the other is Unassigned, then the one that is Unassigned is now forced to be whatever value that will make it evaluate to True. Why? Because whenever a watched literal evaluates to False, no matter what, we try to find another literal inside the clause that can become a new watched literal. If no such replacement exists, then that watched literal is forced to stay False within the clause. Therefore, if the other is Unassigned, it must be forced to evaluate to True, and if the other is False, then that means that not only are there no replacements, but there's a forced contradiction, so we have to backtrack.

As an example: INSERT EXAMPLE YES IM LAZY

To implement this algorithm, we need to make some architectural changes to our current DPLL. Firstly: no more pure literal elimination. We could technically keep it, but it adds an extra layer of complexity and it didn't really help runtime much. If you're implementing your own solver and want to keep it, that's fine, but I'm removing it from here on out. Second: no more *satisfied* tracking because it's also an extra complexity layer and believe me, when you see the results, you won't really care for it either. Third: no more *learned* literal tracking because yes it's an extra layer of complexity, but more importantly, we're moving away from literals being assigned sequentially. In our current implementation, in order to keep track of True, False, or Unassigned, we need to check the absolute value of the literal, compare it to the size of the current assignment vector, and then from there can we infer if it was forced to be whatever it was, or if it was inferred from something else, and that can get confusing fast. Instead we'll just use the values for 0 = False, 1 = True, and 2 = Unassigned (that way, a False check is just `!lits[i]`). So now we will modify our `solve()` methods:

```
SHOW PSEUDOCODE OF SOLVECNF() AND SOLVE()
THEN INSERT EXPLANATION
INSERT NEW API
NOW INSERT PSEUDOCODE OF PROPAGATE()
INSERT EXPLANATION
```

6.1 Watched Literals - Results

After implementing watched literals, removing pure literal elimination, and satisfied clause tracking, our time to complete our test suite dramatically increases. For our unsatisfiable test suite, it took 6065ms to solve all 1000 problems. No I didn't miss a digit anywhere. It took basically 6 seconds to solve the same 1000 problems that initially took us over half an hour to solve. In our optimized DPLL, we were able to bring it down to 85000ms, which is just under 2 minutes. So from 2 minutes to 6 seconds is a 20x improvement in speed. Next, for our satisfiable problems, it took only 352ms to solve all 1000 problems. So that was almost a 4x improvement from last time (1103ms).

7 Clause Learning - The Beginning of CDCL

Clause learning is the foundation of modern SAT solving. If you thought watched literals was an improvement, you haven't seen anything yet. Conflict Driven Clause Learning, or CDCL, is about changing the search space. Instead of simply exploring and pruning branches from the decision tree, which may get re-explored from time to time to some degree, CDCL is about figuring out the relationship between literals and what must be true between them. For instance, if $-1 = 4 = -5$ contradiction, then it would be useful to remember that those literals together can't ever take on those values for the future.

7.1 Iterative DPLL

The first step in our migration from DPLL to CDCL is to rewrite DPLL as an iterative algorithm as opposed to a recursive one. This is because CDCL relies on new ways of keeping track of what we've solved in our equation. Things such as decision levels, implication tracking, and branch flipping need to be formalized outside of the implicit method of tracking we've been using with our old data structures. These new data structures will allow us major improvements in the future such as non-chronological backtracking, allowing us to undo multiple decisions at once. Furthermore, once we implement clause learning, it doesn't make sense to traverse the "decision tree" anymore as clause learning modifies the search space in a way that isn't well represented by a decision tree, and as such, recursion would become too much of a hassle to maintain. And if none of these reasons are enough, recursive algorithms are slower than iterative algorithms, and since the goal is to solve these equations as fast as possible, we want to make speed gains anyways. ²

Because I'm lazy and this is not the final version of the paper (and no one is looking at my GitHub repo anyways), I am not going to write out the implementation differences until the final version of the paper. Instead I will describe the data structures so future me can remember what's going on.

The first thing we need to keep track of are *decisionlevels*, these include things such as: literal being assigned/learned, as well as the clause that "forced" that decision. When that decision clause is *null*, we immediately know that we made a decision, or forced that literal to be learned. We want to keep an array

²I am an avid hater of recursion and "elegant code" or "more readable" code is not sufficient enough reason to implement something recursively to me.

of these decision levels. Next, we want to know when one decision level ends and the next one starts. As defined above, a decision level starts when the reason clause is *null*. When it's not *null*, it's because that decision we made propagated out to other literals, so they are *inferred*. So in order to keep track of where each decision level is, we just keep track of the indices of each entry with a null reason clause. We will keep a separate array of these markers.

In our *solveCNF()* method, we remove the call to *solve()* as it's no longer needed. After we create our watched literals, we enter an infinite loop to try to solve the equation. Looking at the code for this, you'll see we use a *while(true)* loop, why not a *for()* loop that's guaranteed to terminate? That's because it's hard to quantify exactly how many iterations it will take exactly. Of course, with clever indexing incrementing and decrementing, the exact number of iterations wouldn't exactly matter, and you *could* represent the loop that way instead, but there isn't much benefit to doing it that way.

Inside the infinite loop, we immediately call the propagate method, which has been slightly changed. Now instead of returning a delta vector of changes it's made, it will now return nothing when it runs into a contradiction (in the future, we actually will want it to return the clause that has a contradiction so we can learn it, but that's a future topic). In the case where we run into a contradiction, we try to backtrack. While we have at least one decision level left, we unassign each literal in that level until we reach the beginning of that level. If we have not tried assigning that literal to be *false* (since we initially force it true), we will retry that entire decision level with the current decision as *false* instead of *true*. If we *have* tried it, we undo that decision entirely, remove it from our decision level array, and then we evaluate the next most recent decision. We do this until we have reached a decision level where we have not forced the decision *false*, or until there are no more decision levels left to undo. In the case where we have undone everything, we return that the equation is unsatisfiable. Otherwise, we continue onto the next iteration of the loop.

After we have taken care of the contradiction, or if there is no contradiction to begin with, we then check for the next unassigned literal. If there is none, then we have found a satisfiable solution for our CNF equation, and we can terminate the loop, otherwise, we force the first unassigned literal to be *true*, add in our new decision level and decision marker, and we mark this "qhead", which is the starting point of propagation. "qhead" is an index variable pointing to the most recent decision level made and is used in the changed *propagate()* method which we will be discussed now.

Luckily, most of the logic is going to be unchanged. For a refresher, the way *propagate()* worked, is that we would pass in a literal, add it into a queue, then while the queue wasn't empty, we'd deque (remove the first element) and then perform some form of unit propagation. Since we added watched literals, we would only perform this on certain clauses, but if we found a new unit literal, we'd add it to the queue and repeat until we either empty the queue (and return the delta vector), or find a contradiction (and undo what we've inferred). Now that we're working with decision levels which implicitly store what we've learned, we need to change our approach.

As said before, qhead is an index variable pointing to the most recent decision made. The new condition is no longer "until the queue is empty," it's "until qhead points to the end of the decision trail." If there is nothing to do unit

propagation on, then the loop will run for just one iteration. The algorithm remains mostly the same: we grab the current literal, in this case the literal at the index of `qhead`'s value, increment `qhead` (so no infinite loops), perform unit clause propagation on the negative of the current literal (because of watched literals), and then if we find a literal that we can infer, we make a new decision level (the data structure, not one we index), and add it to the decision trail. This data struct contains the literal and the clause that allowed it to be inferred (the one where every other literal is false except for the one we just inferred).

With that, we have implemented iterative DPLL. You may be curious as to what the results are, and on our current test cases, there was not a significant difference in speed up, just a few milliseconds each. But like I said, CDCL (more or less) requires an iterative approach, so our efforts won't go to waste in the next section.

8 Clause Learning